

8h às 10h

Autoestudos Principais

1

O que é hiperparâmetro ?

2

Validação Cruzada

3

Como escolher um modelo preditivo

4

Explicabilidade de Modelo com SHAP

| Stand-up daily: em pé, direto ao ponto

15 min

10:00 → 10:15

1

Em pé

A daily é rápida — todos de pé.

2

Tasks do projeto

O que cada um fez e o que falta.

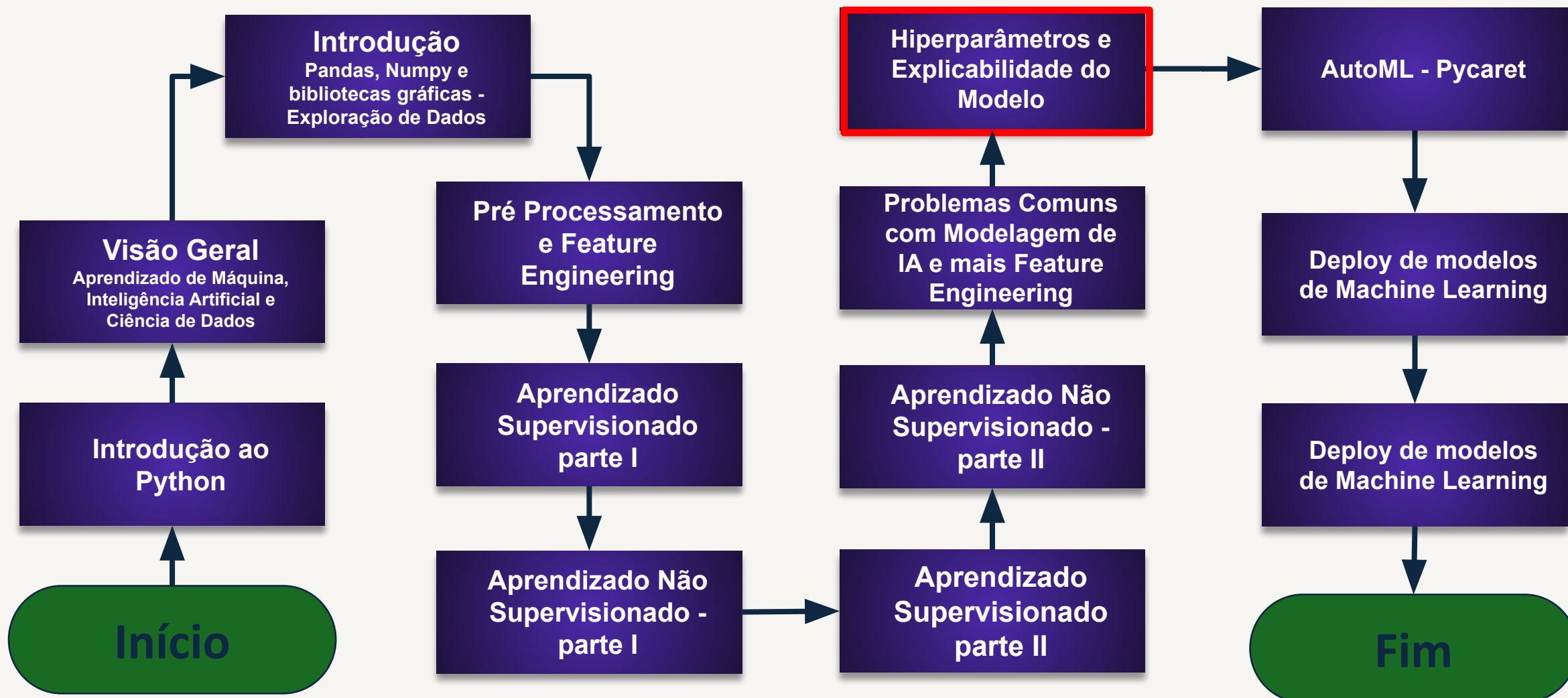
3

Cronograma do dia

Definir prioridades e metas de hoje.

Objetivo: alinhar o time e destravar o dia.

DIAGRAMA DO MÓDULO



AULA 10 • SEMANA 7

Hiperparâmetros e Explicabilidade do Modelo

Validação Cruzada • Hiperparâmetros • Grid Search • Random Search • SHAP

Prof^a Dra. Crishna Irion

Agenda do Encontro

1

Conceito

5 blocos: da escolha do modelo à busca por hiperparâmetros e explicabilidade

2

Aplicação prática

Como escolher um modelo preditivo pelo contexto do problema

Validação Cruzada: hold-out vs K-Fold

O que é um hiperparâmetro

Grid Search e Random Search

Explicabilidade de modelo: SHAP

Contextualizando a necessidade de explicabilidade

3

Código

Validação cruzada com `cross_val_score` e `KFold`

Busca de hiperparâmetros com `GridSearchCV`

Busca de hiperparâmetros com

`RandomizedSearchCV`

Explicabilidade com `shap.Explainer`

PARTE 1

Como Escolher um Modelo Preditivo

Entender que a escolha do melhor modelo depende do contexto do problema — e não só da métrica mais alta.

Como já vimos na aula passada, a acurácia sozinha não conta a história toda.

```
Acuracia do modelo: 99%  
990 de 1000 casos sao "nao fraude"  
So 10 sao "fraude" - e o modelo erra os 10  
Aprovamos esse modelo assim mesmo?
```

Bem na média, mal onde importa.

O modelo aprendeu a distinguir as classes ou só a prever a classe majoritária?
Qual métrica realmente importa para esse problema?

A escolha depende da natureza dos dados (numéricos, texto, imagens) e do objetivo do problema (classificação ou regressão).

Deve-se buscar o equilíbrio entre a simplicidade do modelo e sua capacidade de aprendizado, evitando o subajuste (underfitting) ou o sobreajuste (overfitting).

Duas prioridades em conflito: Precisão x Recall

PRECISÃO (previsão positiva)

- das previsões positivas, quantas realmente são?
- importa quando o custo de um falso positivo é alto
- ex: marcar um e-mail bom como spam
- modelo conservador: só marca positivo com certeza

RECALL (caso positivo real)

- dos casos positivos reais, quantos o modelo capturou?
- importa quando o custo de um falso negativo é alto
- ex: deixar passar um câncer ou uma fraude
- modelo sensível: prefere alertar demais a deixar passar

**Não existe modelo perfeito nas duas métricas
— a escolha depende do problema.**

Como decidir: 3 perguntas para escolher seu modelo

1

Qual o custo do erro?

falso positivo e falso negativo custam o mesmo? priorize precisão ou recall de acordo

2

Há desbalanceamento?

acurácia engana quando uma classe domina — olhe F1, precisão e recall

3

Preciso explicar a decisão?

em contextos sensíveis (crédito, saúde), um modelo simples e explicável pode valer mais que 1% de métrica

Não existe "o melhor modelo" — existe o modelo certo para o seu contexto.

PARTE 2

Validação Cruzada

Ir além do hold-out: avaliar o modelo de forma mais robusta, sem depender de uma única divisão dos dados.

Validação cruzada, o que é?

É uma técnica para avaliar a capacidade de generalização de um modelo.

Ela divide os dados em várias partes (dobras ou folds), treinando o modelo em algumas e testando em outras de forma rotativa.

Isso garante que o modelo seja testado em todo o conjunto de dados, gerando uma métrica de desempenho mais confiável e realista.

Método Hold out - é uma técnica que envolve dividir seus dados em dois subconjuntos: um para treinar e outro para testar.

O conjunto de treinamento é usado para ajustar os parâmetros do modelo, enquanto o conjunto de teste é usado para avaliar o quão bem o modelo prevê dados não vistos.

Hold-out: a forma mais simples de validar

o problema

depende de qual parte dos dados caiu no treino e qual caiu no teste

por que atrapalha

- 1 · o resultado muda conforme a divisão sorteada
- 2 · uma divisão "sortuda" pode superestimar o desempenho

A alternativa: K-Fold

Com poucos dados, uma única divisão pode não representar bem a variabilidade real do problema. K-Fold treina e testa em K partições e tira a média das rodadas. Assim, todos os exemplos são testados — em rodadas diferentes.

Ponto de atenção: hold-out ainda serve para uma checagem rápida, mas validação cruzada é mais confiável para comparar modelos.

Hold-out vs K-Fold: duas formas de validar

HOLD-OUT

"divida uma vez em treino e teste"

rápido e simples de aplicar
depende de qual parte caiu em cada conjunto
risco: uma divisão sortuda distorce a avaliação

K-FOLD CROSS VALIDATION

"divida em K partes e revezei quem é teste"

cada parte vira teste uma vez; o resto vira treino
a média das K rodadas é mais estável
risco: custo computacional K vezes maior

Código: validação cruzada com SCIKIT-LEARN



1. K-Fold com `cross_val_score`

Passa o modelo, X, y e o número de folds (cv).

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(modelo, X, y, cv=5)
print(scores)
# [0.81, 0.79, 0.83, 0.80, 0.82]
```



2. Média e desvio padrão

A média resume o desempenho esperado; o desvio mostra a estabilidade.

```
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(modelo, X, y, cv=kf)
print(scores.mean(), scores.std())
# 0.81 (+/- 0.014) -> desempenho estavel entre os folds
```

`cross_val_score` e `KFold` automatizam dividir, treinar e avaliar em várias rodadas.

PARTE 3

Hiperparâmetros

Entender a diferença entre parâmetro e hiperparâmetro — e por que ajustá-los importa.

Parâmetro x Hiperparâmetro: qual a diferença?

k = ?

Hiperparâmetro é um valor que você escolhe antes de treinar o modelo.

Exemplos: taxa de aprendizado, o número de árvores em uma floresta aleatória ou a profundidade máxima de uma árvore de decisão.

1

Parâmetro

aprendido pelo próprio algoritmo durante o treino (ex.: pesos de uma regressão)

2

Hiperparâmetro

definido por você antes do treino (ex.: k do KNN, profundidade da árvore)

3

Por que testar valores

o valor certo depende dos dados — não existe um número mágico universal

Hiperparâmetros na prática: exemplos por algoritmo

1

KNN

n_neighbors: número de vizinhos —
poucos: sensível a ruído;
muitos: perde detalhes

2

Árvore de Decisão

max_depth: profundidade máxima —
 rasa: underfitting; funda: overfitting

3

SVM

C e kernel: controlam a margem —
quanto o modelo tolera erros no treino

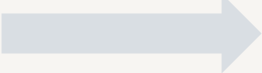
Cada algoritmo tem seus próprios hiperparâmetros — não existe um conjunto universal ideal.

PARTE 4

Grid Search e Random Search

Sistematizar a busca pelos melhores hiperparâmetros — testando tudo ou testando por amostragem.

Grid Search: testar todas as combinações da grade

GRID  **MELHOR**

- cria uma grade com todos os valores possíveis de cada hiperparâmetro
- testa cada combinação da grade, uma por uma
- para cada combinação, avalia com validação cruzada
- escolhe a combinação com melhor média de desempenho

Grid Search é exaustivo — cresce rápido conforme mais hiperparâmetros e valores entram na grade.

Grid Search vs Random Search: buscar tudo ou buscar por amostragem

GRID SEARCH

testa todas as combinações da grade
garante cobrir cada combinação possível
cresce exponencialmente com mais
hiperparâmetros
indicado quando a grade é pequena

RANDOM SEARCH

sorteia combinações aleatórias da grade
testa um número fixo de combinações
(n_iter)
encontra boas soluções com menos
tentativas
indicado quando a grade é grande

Random Search costuma achar uma combinação quase tão boa quanto o Grid Search, com muito menos tentativas.

Código: busca de hiperparâmetros com GridSearchCV

1. Grade de hiperparâmetros

Um dicionário com os valores que você quer testar para cada hiperparâmetro.

```
from sklearn.neighbors import
KNeighborsClassifier
print(param_grid)
```

2. Rodar a busca com validação cruzada

GridSearchCV testa cada combinação com cross-validation e guarda a melhor.

```
from sklearn.model_selection import GridSearchCV
param_grid = {'n_neighbors': [3,5,7,9],
'weights': ['uniform','distance']}

grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)

# melhor combinacao encontrada:
print(grid.best_params_)

# {'n_neighbors': 7, 'weights': 'distance'}
```

Código: busca de hiperparâmetros com RandomizedSearchCV



1. Definir a distribuição de valores

Pode usar listas ou distribuições contínuas (ex.: `scipy.stats`).

```
from sklearn.model_selection import
RandomizedSearchCV
param_dist = {'n_neighbors': range(1,30),
'weights': ['uniform','distance']}
```



2. Rodar com `n_iter` combinações

`n_iter` define quantas combinações aleatórias serão testadas.

```
from sklearn.neighbors import KNeighborsClassifier
random_search = RandomizedSearchCV(KNeighborsClassifier(),
param_dist, n_iter=20, cv=5, random_state=42)
random_search.fit(X_train, y_train)

print(random_search.best_params_)

# {'n_neighbors': 11, 'weights': 'distance'}
```

Random Search sorteia um número fixo de combinações da grade, em vez de testar todas.

PARTE 5

Explicabilidade do Modelo com SHAP

Entender por que o modelo decide o que decide — e quando essa explicação realmente importa.

Por que abrir a caixa-preta? Depende do contexto

o problema

exigir explicação de todo modelo, sempre, do mesmo jeito

por que atrapalha

- 1 · em baixo risco, pode limitar benefícios sem necessidade
- 2 · em alto risco (crédito, saúde), a falta dela pode ser inaceitável

O que considerar

A explicabilidade deve depender do contexto e do impacto potencial da decisão — não é uma exigência universal para todos os sistemas de IA. Avalie caso a caso: quem é afetado e o que a explicação muda na prática.

Ponto de atenção: exigir explicabilidade sempre tem custo — em modelos de baixo risco, pode limitar ganhos de desempenho sem necessidade real.

SHAP: como funciona a explicação por valores de Shapley

1

Valor base

a previsão média do modelo, sem olhar nenhuma variável específica

2

Contribuição de cada variável

cada variável empurra a previsão para cima ou para baixo

3

Predição final

valor base + soma das contribuições = previsão do modelo

SHAP não diz só "o que" o modelo previu — mostra "por que", variável por variável.

Código: explicabilidade com SHAP



1. Criar o explainer

Passe o modelo treinado e uma amostra dos dados de referência.

```
import shap
explainer = shap.Explainer(modelo, X_train)
shap_values = explainer(X_teste)
# shap_values guarda a contribuicao de
# cada variavel em cada previsao
```



2. Visualizar as contribuições

summary_plot mostra, para cada variável, o impacto médio nas previsões.

```
shap.summary_plot(shap_values, X_teste)
# cada ponto = 1 previsao; cor = valor da variavel
```

shap.Explainer calcula a contribuição de cada variável para cada previsão.

PARTE 6

Mão na Massa

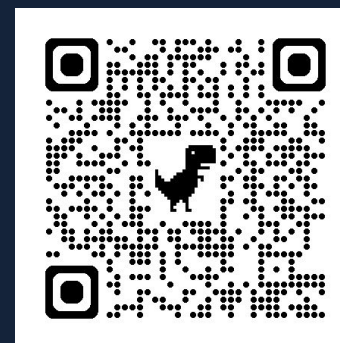
Dinâmica SHAP



PARTE 6

Mão na Massa

Código

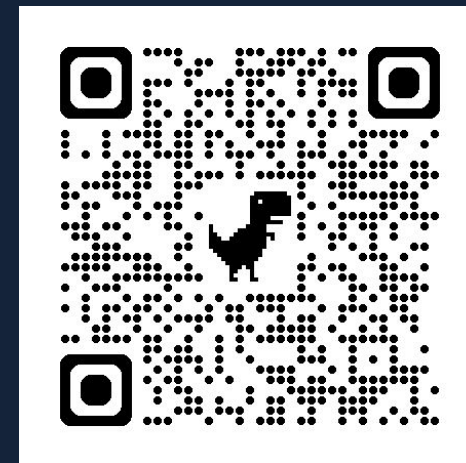


<https://encurtador.com.br/meAI>

PARTE 6

Mão na Massa

Gabarito do Código



<https://encurtador.com.br/EmrQ>

Dúvidas? Vamos conversar.

Profª Dra. Crishna Irion

crishna.irion@prof.inteli.edu.br

